

GUÍA PRÁCTICA AVANZADA

Kotlin esencial y dominio KMP

Modelado de dominio, null-safety, data classes y pruebas en PharmaMobile

Estudiante	Roberto Samuel Valencia Saavedra
Docente	Benjamin David Reyna Barreto
Proyecto	PharmaMobile - Kotlin Multiplatform
Repositorio	github.com/Akatosh4019/PharmaMobile_Valencia

1. Propósito y alcance

La práctica consolida el modelado de dominio compartido en commonMain mediante entidades inmutables, manejo seguro de datos anulables, relaciones entre objetos y validaciones de negocio. Las evidencias corresponden al proyecto PharmaMobile desarrollado con Kotlin Multiplatform.

2. Organización del dominio

Los modelos Cliente, Producto, Pedido, DetallePedido y EstadoPedido se encuentran en el paquete `pe.edu.upeu.pharmamobile.domain.model`, dentro del código compartido. Esta ubicación permite reutilizar la lógica de negocio desde Android e iOS.

3. Evidencias de implementación

Las siguientes capturas documentan la implementación realizada en Android Studio y la validación de una prueba local de Kotlin, sin depender de un emulador.

3.1 Entidad Cliente y null-safety

La entidad Cliente utiliza identificador, nombre y correo obligatorios, mientras que telefono es anulable. El método `obtenertelefono()` aplica el operador Elvis para devolver un mensaje seguro cuando no existe un número registrado.

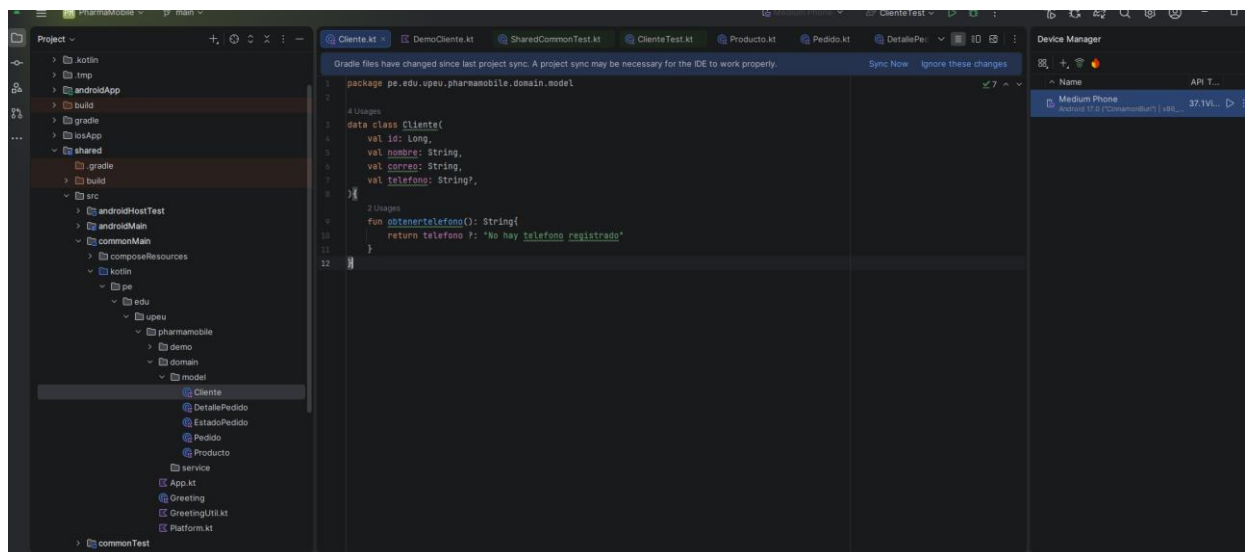


Figura 1. Entidad Cliente y null-safety

3.2 Entidad Producto

Producto se modela como data class con id, nombre, precio y stock. La inmutabilidad de sus propiedades facilita el uso de `copy()` para producir nuevas versiones sin modificar la instancia original.

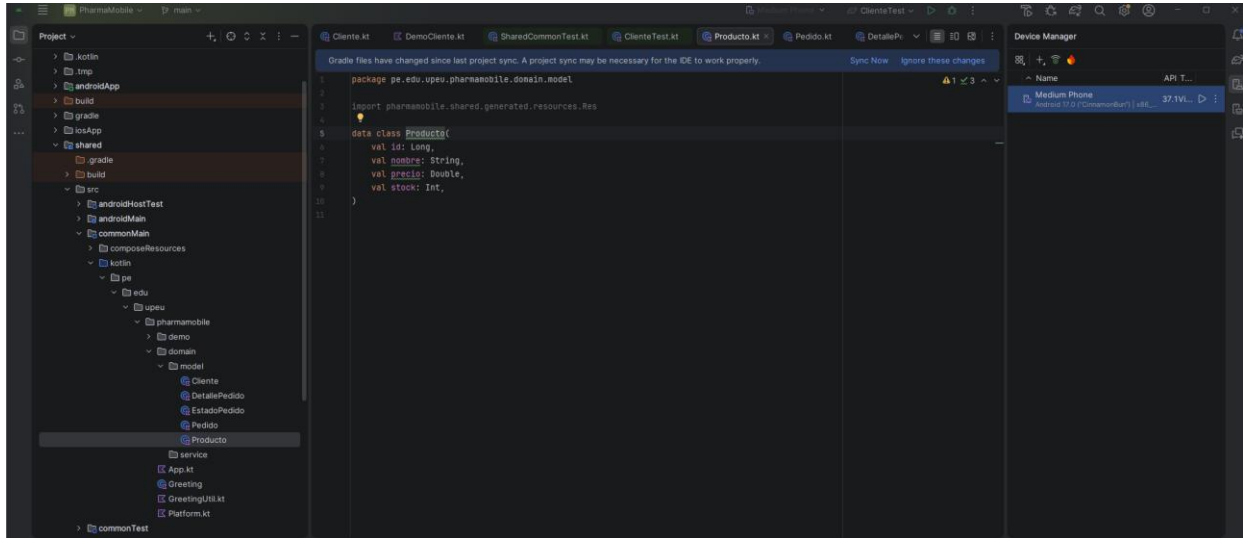


Figura 2. Entidad Producto

3.3 Entidad Pedido y relaciones del dominio

Pedido integra las referencias a Cliente y Producto, además de cantidad, fecha, estado y total. De esta forma se representa la asociación entre la persona que compra y el producto solicitado.

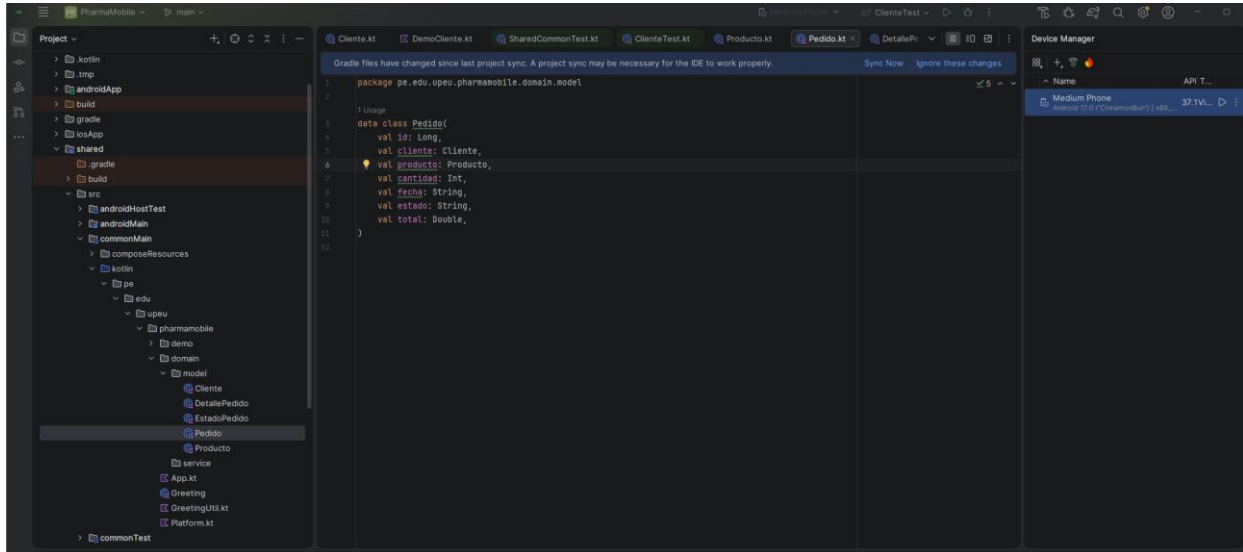


Figura 3. Entidad Pedido y relaciones del dominio

3.4 DetallePedido y validaciones

DetallePedido encapsula la cantidad y el precio unitario de una operación. El bloque init impide cantidades menores o iguales a cero y precios negativos; el subtotal se obtiene de manera calculada.

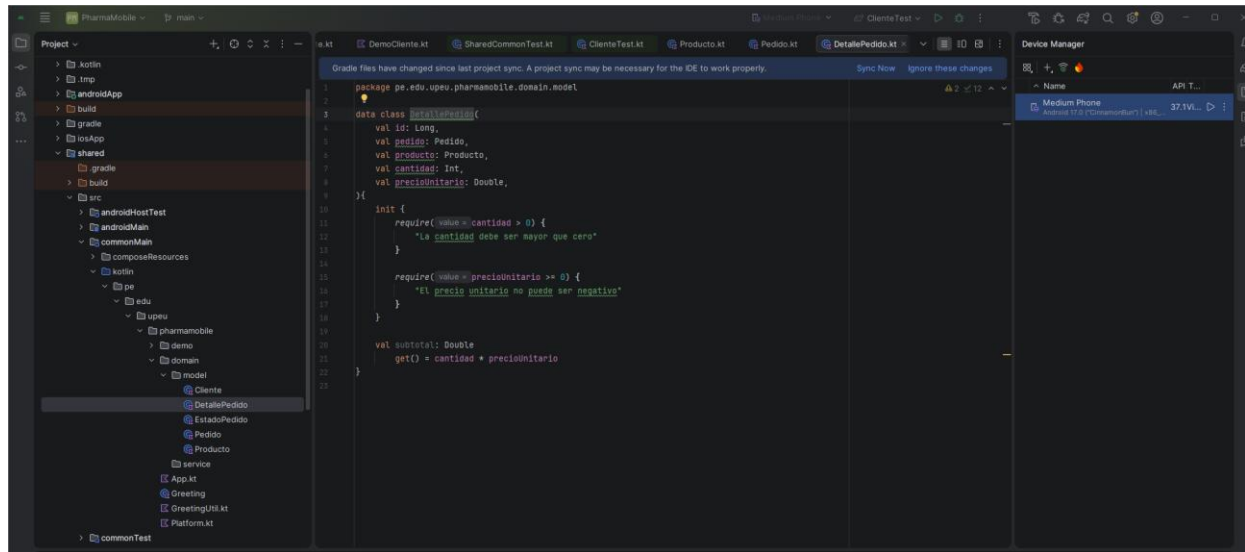


Figura 4. DetallePedido y validaciones

3.5 Estados mediante sealed class

EstadoPedido restringe los estados posibles a Pendiente, Procesando, Entregado y Rechazado. El estado Rechazado puede transportar el motivo, lo cual permite un tratamiento exhaustivo mediante when.

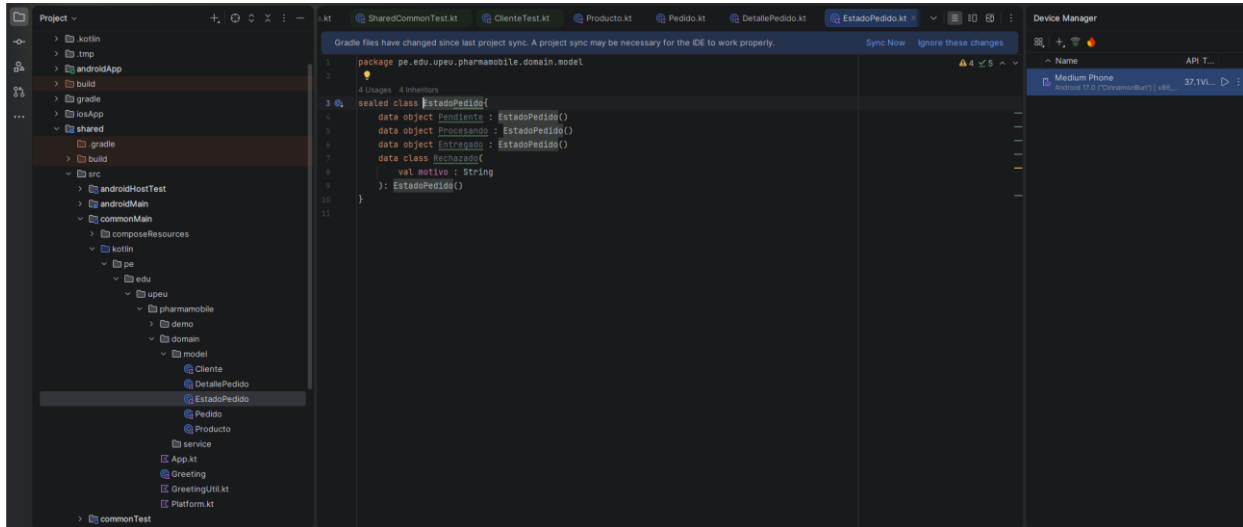


Figura 5. Estados mediante sealed class

3.6 Prueba de Cliente

ClienteTest comprueba el comportamiento cuando telefono es null. La ejecución local finaliza con una prueba aprobada y BUILD SUCCESSFUL, evidenciando que la lógica compartida funciona sin ejecutar el emulador.

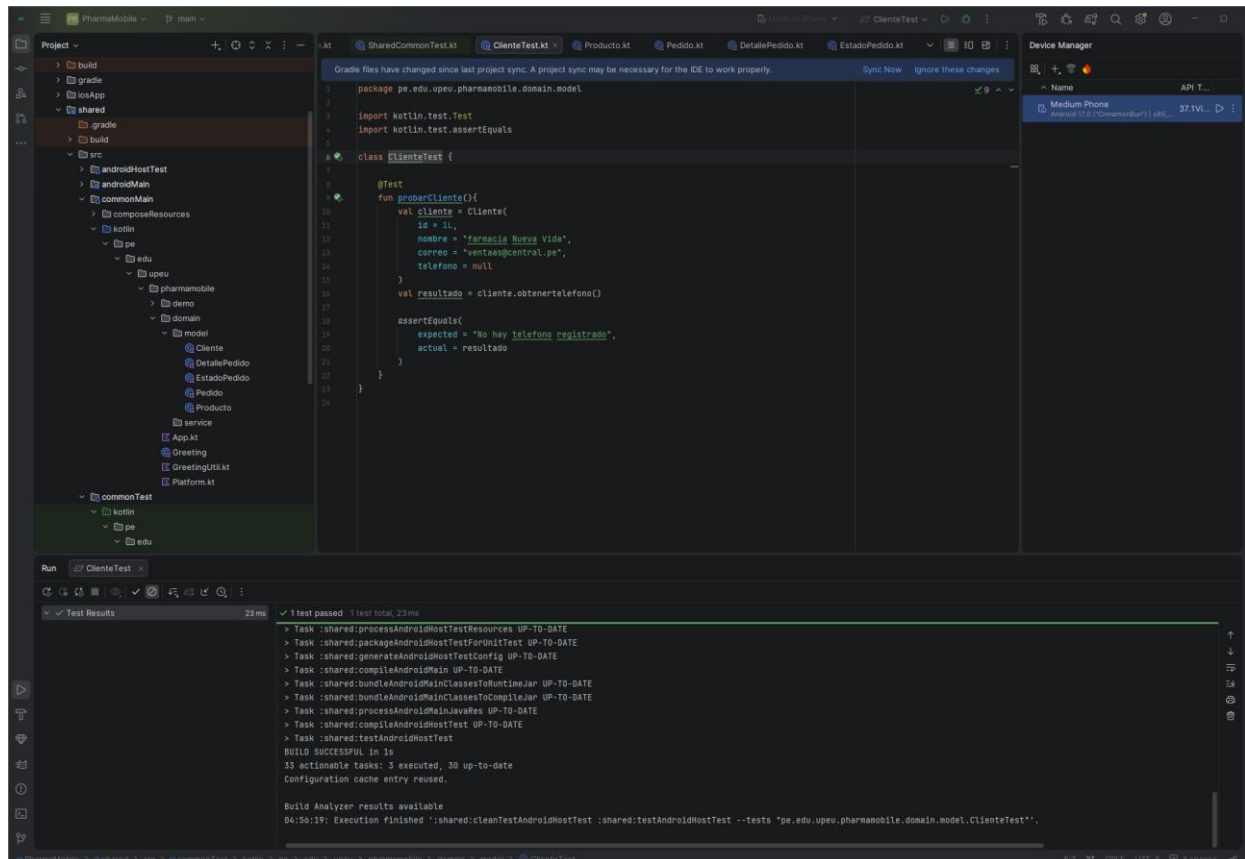


Figura 6. Prueba de Cliente

4. Resultado de la práctica

Se construyó un dominio compartido con entidades representativas de una farmacia. La solución demuestra el uso de data classes, tipos anulables, operador Elvis, validaciones con require, propiedades calculadas y una jerarquía sellada para estados controlados.

5. Conclusiones

- El código de dominio permanece en commonMain y puede reutilizarse entre plataformas.
- La null-safety evita errores al consultar información opcional del cliente.
- Las validaciones de DetallePedido protegen reglas básicas antes de procesar una operación.
- La sealed class EstadoPedido delimita los estados válidos y admite información asociada a un rechazo.
- La prueba ClienteTest se ejecutó como prueba local y terminó correctamente.

6. Repositorio

El código fuente y el historial del proyecto están disponibles en:

https://github.com/Akatosh4019/PharmaMobile_Valencia